

Practical #8

Practical Title:

To design and execute automated test scripts using Selenium for functional testing

Theory:

1. What is Functional Testing?

Functional testing is a type of testing where we verify that the application works according to the given requirements.

👉 It checks:

- Login functionality
- Form submission
- Buttons, links, and features

2. Advantages of Selenium for Testing

Using Selenium:

- Supports multiple browsers (Chrome, Firefox, Edge)
- Supports multiple programming languages
- Open-source and free
- Easy integration with CI/CD tools
- Automates repetitive testing tasks

Procedure:

Step 1: Setup Selenium environment

Step 2: Launch browser using WebDriver

Step 3: Navigate to a web application

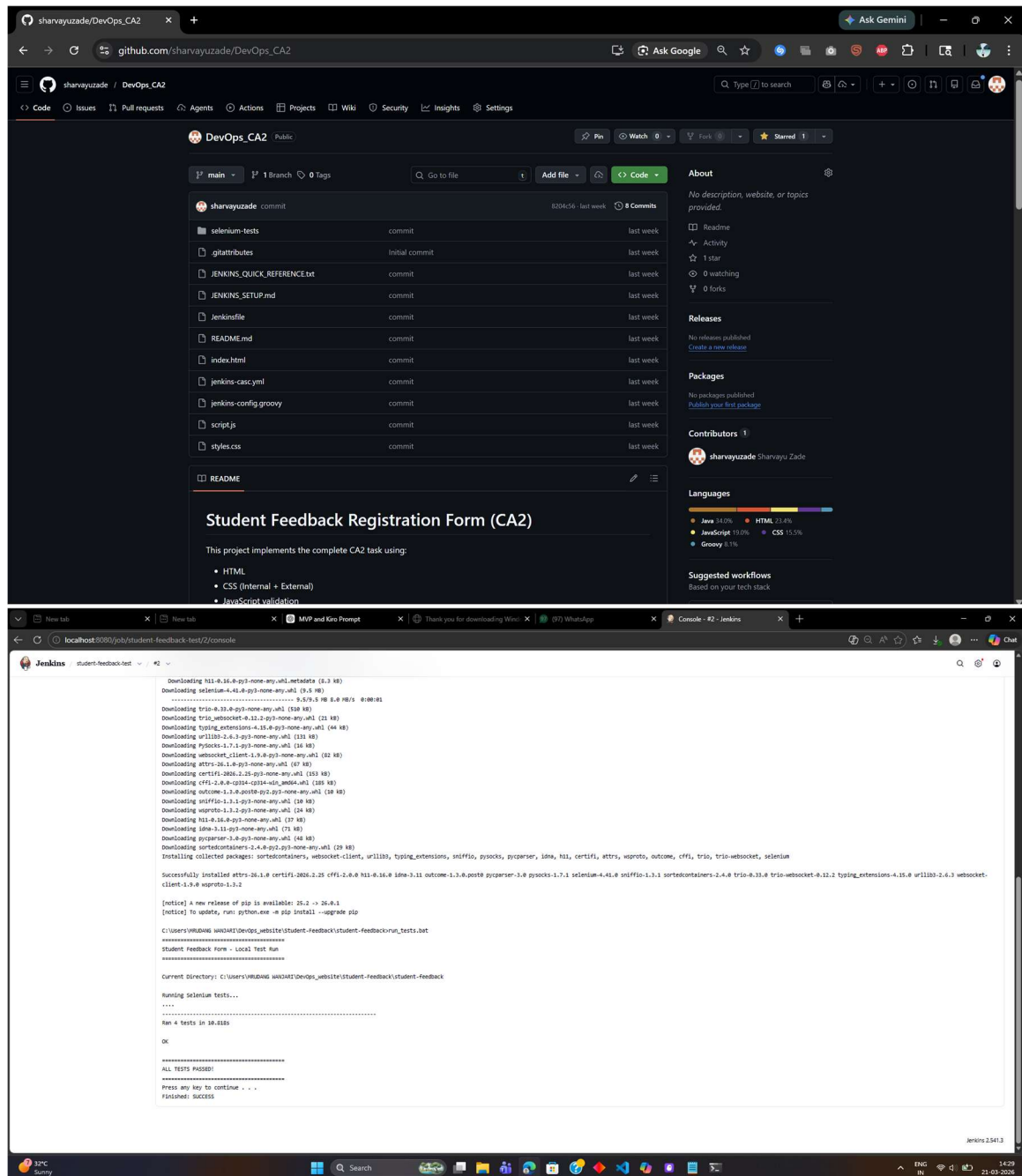
Step 4: Locate elements using locators (id, name, xpath, etc.)

Step 5: Perform actions (click, send keys, submit)

Step 6: Validate output using assertions

Step 7: Close the browser

Output Screenshots:



Activity / Task:

Task 1: Selenium Login Test Script

Code (Python Example):

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
import time

driver = webdriver.Chrome()

# Open demo login page
driver.get("https://the-internet.herokuapp.com/login")

# Enter username and password
driver.find_element(By.ID, "username").send_keys("Mrudang")
driver.find_element(By.ID, "password").send_keys("SuperSecretPassword!")

# Click login
driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

# Validate login success
time.sleep(2)
assert "secure area" in driver.page_source.lower()

# Close browser
driver.quit()
```

Task 2: Form Testing (Valid & Invalid Inputs)

Description:

- **Valid Input Test:**
 - Enter correct data (username, password)
 - Submit form
 - Verify success message
- **Invalid Input Test:**
 - Enter wrong data
 - Submit form
 - Verify error message

This ensures:

- Form validation works properly
 - Error handling is correct
-

Advantages

1. Faster execution than manual testing
 2. Supports automation of functional tests
 3. Reusable scripts
 4. Cross-browser testing
 5. Integration with DevOps pipelines
-

Challenges

1. Requires coding knowledge
2. Maintenance of scripts
3. Handling dynamic elements can be difficult